Blinkit Data Analysis with SQL

This project focuses on comprehensive data analysis of Blinkit, an online grocery platform, utilizing SQL to extract meaningful insights.

By
Sathish Kumar Ravichandran

BLINKIT DATA ANALYSIS - MY SQL

1. Customer Insights

1.1 Identify the top 10 customers by total order value.

```
SELECT
    c.customer_name,
    ROUND(SUM(o.order_total), 2) AS total_order_value
FROM
    customers c
JOIN
    orders o ON c.customer_id = o.customer_id
GROUP BY
    customer_name
ORDER BY
    total_order_value DESC
LIMIT 10;
```

1.2 Count the number of customers in each segment (Premium, Regular, Inactive, New).

```
SELECT
    customer_segment,
    COUNT(*) AS no_of_customers
FROM
    customers
GROUP BY
    customer_segment;
```

1.3 Find customers with an average order value above 500 and more than 10 orders.

```
SELECT
    c.customer_name,
    AVG(o.order_total) AS avg_order_value,
    COUNT(o.order_id) AS total_orders
FROM
    customers c
JOIN
    orders o ON c.customer_id = o.customer_id
GROUP BY
    c.customer_name
HAVING
    avg_order_value > 500
    AND total_orders > 10;
```

2. Order and Delivery Performance

2.1 List orders that were delivered late along with reasons for delay.

```
SELECT
    order_id,
    delivery_status
FROM
    delivery_performance
WHERE
    delivery_status <> 'on time';
```

2.2 Calculate the average delivery time (in minutes) per delivery partner.

```
SELECT
    delivery_partner_id,
    AVG(delivery_time_minutes) AS avg_delivery_time
FROM
    delivery_performance
GROUP BY
    delivery_partner_id;
```

2.3 Find the top 5 stores by total order revenue.

```
WITH a AS (
   SELECT
      o.store_id,
      SUM(oi.unit_price * oi.quantity) AS order_revenue
   FROM
      orders o
   JOIN
      order_items oi ON o.order_id = oi.order_id
   GROUP BY
      store_id),
b AS
(SELECT
      *,
      DENSE_RANK() OVER (ORDER BY order_revenue DESC) AS rak
   FROM
      a)
SELECT
   store_id,
   order_revenue
FROM
   b
WHERE
   rak <= 5;
```

3. Product and Inventory Analysis

3.1 Identify products that have had damaged stock more than 5 times in total.

```
SELECT
    p.product_name,
    COUNT(inew.damaged_stock) AS damage_events
FROM
    products p
JOIN
    inventory inew ON inew.product_id = p.product_id
WHERE
    inew.damaged_stock > 0
GROUP BY
    p.product_name
HAVING
    COUNT(*) > 5;
```

3.2 Calculate the total quantity ordered for each product.

```
SELECT
    p.product_name,
    SUM(oi.quantity) AS total_quantity
FROM
    products p
JOIN
    order_items oi ON oi.product_id = p.product_id
GROUP BY
    product_name;
```

3.3 Find products that often fall below the minimum stock level.

```
SELECT
    p.product_name,
    p.min_stock_level,
    inew.stock_received
FROM
    products p
JOIN
    inventorynew inew ON inew.product_id = p.product_id
WHERE
    stock_received < min_stock_level;
```

4. Marketing Campaign Effectiveness

4.1 Calculate total revenue generated and ROAS for each marketing campaign.

```
SELECT
    campaign_name,
    ROUND(SUM(revenue_generated), 2) AS revenue_generated,
    ROUND(SUM(revenue_generated)/SUM(spend), 2) AS roas
FROM
    marketing_performance
GROUP BY
    campaign_name, roas;
```

4.2 Find the campaign with the highest conversion rate.

```
SELECT
    campaign_name,
    SUM(conversions) / SUM(impressions) AS conversions_rate
FROM
    marketing_performance
GROUP BY
    campaign_name
ORDER BY
    conversions_rate DESC
LIMIT 1;
```

4.3 List all campaigns targeted at Premium customers and their performance metrics.

```
SELECT
    campaign_name,
    target_audience,
    clicks,
    revenue_generated,
    roas
FROM
    marketing_performance
WHERE
    target_audience = 'premium';
```

5. Customer Feedback Analysis

5.1 Count feedback entries by sentiment.

```
SELECT
    sentiment,
    COUNT(*)
FROM
    customer_feedback
GROUP BY
    sentiment;
```

5.2 List customers who gave negative feedback and their corresponding orders.

```
SELECT
    c.customer_name,
    o.order_id
FROM
    orders o
JOIN
    customer_feedback cf ON
o.order_id = cf.order_id
JOIN
    customers c ON
c.customer_id =
o.customer_id
WHERE
    sentiment = 'negative';
```